University of Sheffield

Hardware Acceleration for Cluster Fault Tolerance in Hybrid CMOS/non-CMOS Memories

Yuqi Guo 230184273

Supervisor: Mr. Neil Powell

Content

1. Background Introduction
2. Research Objectives
3. Methodology Overview
4. Simulation Results and key finding
5. Conclusion
6. Future Work

Background

- Hybrid CMOS/non - CMOS memories offer ultra - high - density storage, Hybrid memories can achieve >1 Tb/cm² — 10–100× denser than DRAM, but suffer from spatially correlated cluster faults
- Conventional ECCs like RS codes struggle with these faults.
- Redundant Residue Number Systems (RRNS) offer a promising solution by detecting and correcting multiple-bit errors in clusters.
- Existing RRNS implementations are costly or not optimized for memory applications.
- two modified RRNS codes are proposed to tolerate cluster faults in hybrid memories with low cost overhead
    - 3NRM-RRNS
    - 2NMRM-RRNS

RRNS and variants introduction

- **C-RRNS**: Uses a balanced moduli set with redundant moduli slightly larger than non-redundant ones, providing moderate fault tolerance at higher overhead.

- **3NRM-NNRS**: Employs the same non-redundant moduli as C-RRNS but with smaller redundant moduli, reducing complexity while maintaining cluster fault correction capability.

- **2NRM-RRNS**: implementations are costly or not optimized for memory applications.

ECC Types	Moduli	
	Non-Redundant, m_k	Redundant, $m_{(n-k)}$
C-RRNS	{64,63,65}	{67,71,73,79,83,89}
3NRM-RRNS	{64,63,65}	{31,29,23,19,17,11}
2NRM-RRNS	{257,256}	{61,59,55,53}

Research Goals

Algorithm Validation

Evaluate the error correction capabilities of different RRNS schemes and RS.

Hardware Implementation

Design and implement efficient encoder/decoder architectures for all four coding schemes (RS, C - RRNS, 3NRM - RRNS, 2NRM - RRNS) using hardware description languages.

Performance Benchmarking

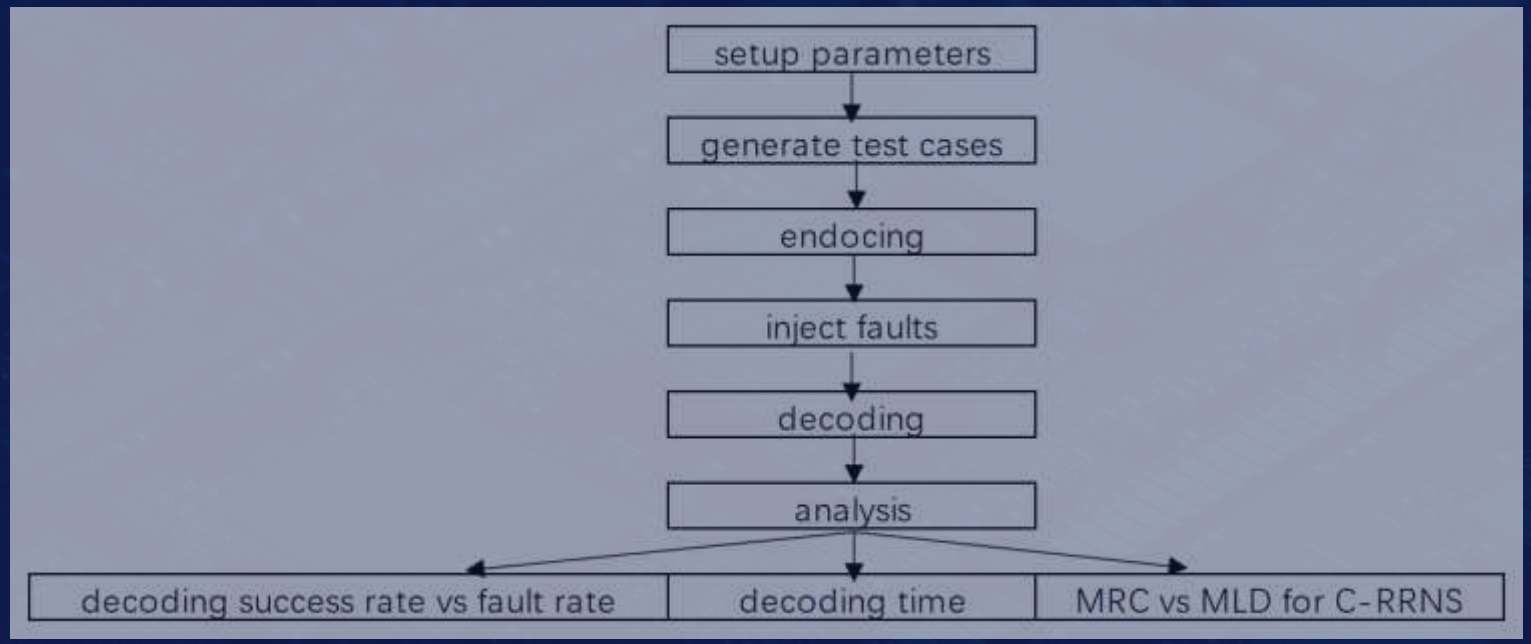
Conduct a comprehensive performance analysis including resource utilization, timing characteristics, and power consumption on FPGA platforms.

Practical Validation

Demonstrate the practical feasibility of the proposed RRNS variants for real - world memory protection applications.

Methodology Overview

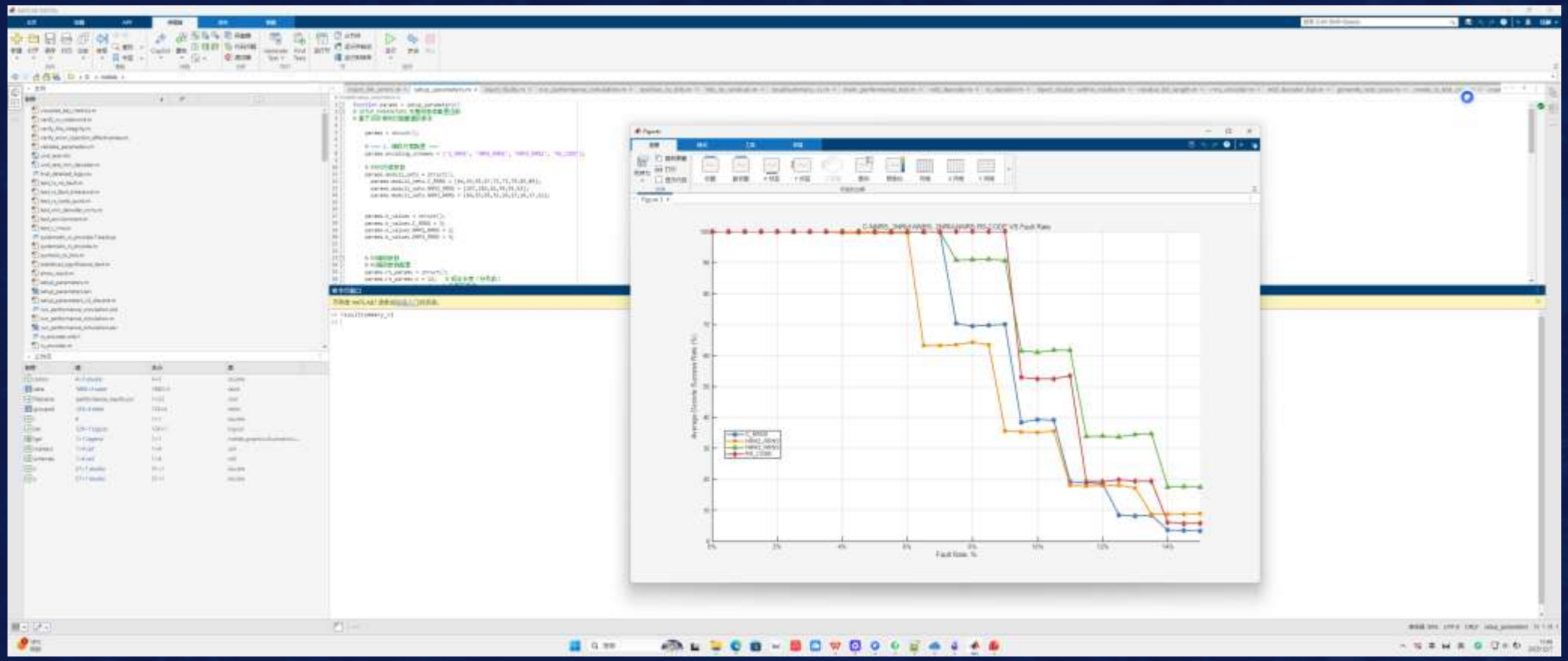
- **Simulation program architecture**



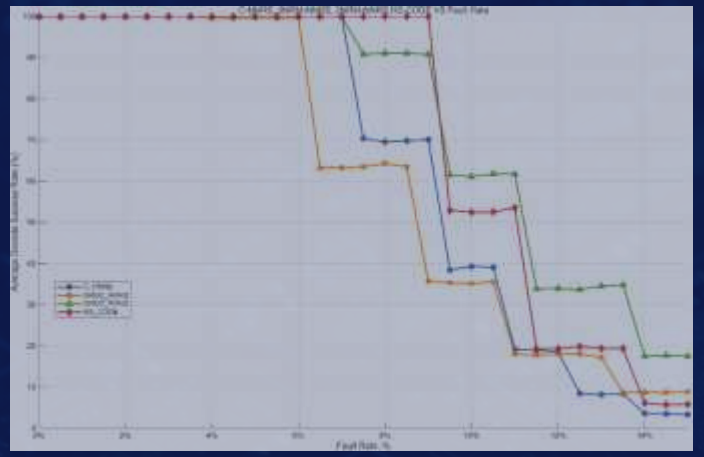
- **Parameters setting**

C-RRNS Moduli	Non-Redundant, mk: {64,63,65} Redundant, m(n-k): {67,71,73,79,83,89}
3NRM-RRNS Moduli	Non-Redundant, mk: {64,63,65} Redundant, m(n-k): {67,71,73,79,83,89}
2NRM-RRNS Moduli	Non-Redundant, mk: {64,63,65} Redundant, m(n-k): {67,71,73,79,83,89}
RS	n = 12, k = 4, t = 4, symbol size m = 4 bits
Number of Monte Carlo trials	1000
Sample data word	Fifteen random integers between 0 and 65,535.

MATLAB Simulation project screenshot



Simulation Result: decoding success rate



C-RRNS decoder: MRC

- **RS_CODE:**

  Exceptional robustness up to 9.5% fault rate; maintains ~6-8% success at 14%.

- **C_RRNS:**

  Optimal for low fault rates (<6%) but sharply degrades beyond 6%.

- NRM2_RRNS:

  Better than C_RRNS in mid-range faults (6–10%), stabilizes at 35–37% success between 8–11%.

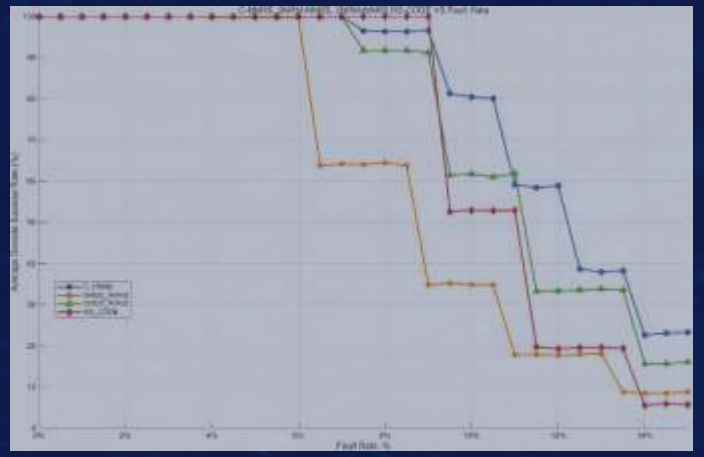
- NRM3_RRNS:

  Outperforms NRM2_RRNS across most fault rates, sustaining over 60% success until 11%.

- Degradation Beyond 12% Fault Rate:

  All codes degrade significantly

Simulation Result: decoding success rate



C-RRNS decoder: MLD

MLD Decoder Impact on C-RRNS:

- **Enhanced High-Fault Performance:**

  Significant improvement in decoding success rate above 10% fault rate; maintains ~38-40% success at 12%, vs. <20% for MRC.

- **Preserves Low-Fault Efficiency:**

  Both versions achieve 100% success below 6% fault rates.

- **Extended Error Correction:**

  More robust decision-making by evaluating all possible values, not just majority voting.

- **Closer to Advanced Schemes:**

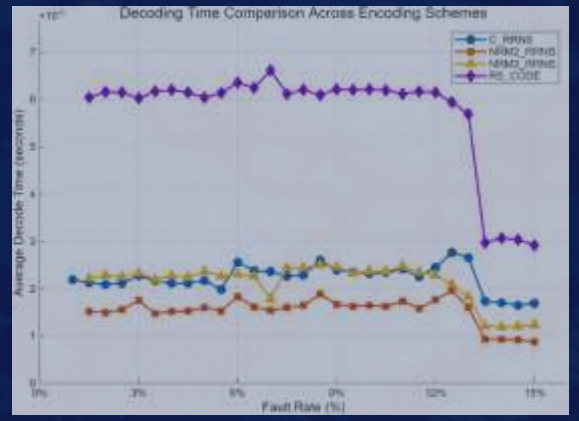
  With MLD, C-RRNS performance approaches that of NRM3_RRNS in the 10–14% fault range.

- **Complexity vs. Reliability Trade-off:**

  Increased computational complexity for higher reliability; suitable for applications needing enhanced fault tolerance.

Simulation Result: decoding Latency

C-RRNS decoder: MRC



C-RRNS decoder: MLD

C-RRNS Decoding Latency: MRC vs. MLD

- MRC: High latency (~10–12 ms), especially at low fault rates — due to complex subset voting.
- MLD: Latency drops to ~2–3 ms across all rates — eliminates redundant enumeration, uses direct likelihood evaluation.

Other schemes (RS, NRM2/3):

- Consistently fast; RS shows near-constant decoding time.

Takeaway:

- MLD makes C-RRNS both faster and more robust, closing the gap with optimized RRNS variants.

Storage Efficiency Comparison

ECC Type	Number of bits			Difference (%)	
	Dataword	Check word	Codeword	Compare with RS	Compare with C-RRNS
RS	4,4,4,4	4,4,4,4,4,4,4,4	48	-	
C-RRNS	6,6,7	7,7,7,7,7,7	61	+27.1	-
3NRM-RRNS	6,6,7	5,5,5,5,5,4	48	0	-21.3
2NRM-RRNS	9,8	6,6,6,6	41	-14.6	-32.8

- **RS_CODE**： uses 48 bits (baseline), but requires high overhead for fault tolerance.
- **C-RRNS**： increases codeword length to 61 bits (+27.1% vs RS), leading to higher storage cost.
- **3NRM-RRNS**： matches RS in total length (48 bits) but reduces redundancy.
- **2NRM-RRNS**： achieves the lowest storage overhead: only 41 bits, saving 14.6% vs RS and 32.8% vs C-RRNS — making it the most compact and efficient option.

Summary of Findings

- **Feasibility:**

  Simulation results confirm that the innovative 3NRM-RRNS and 2NRM-RRNS algorithms proposed in [1] are viable ECC schemes for hybrid CMOS/non-CMOS memories.

- **Fault Tolerance:**

  Across various fault rates, all algorithms provide satisfactory redundancy protection; notably, 3NRM-RRNS offers comparable protection to C-RRNS.

- **Storage Efficiency:**

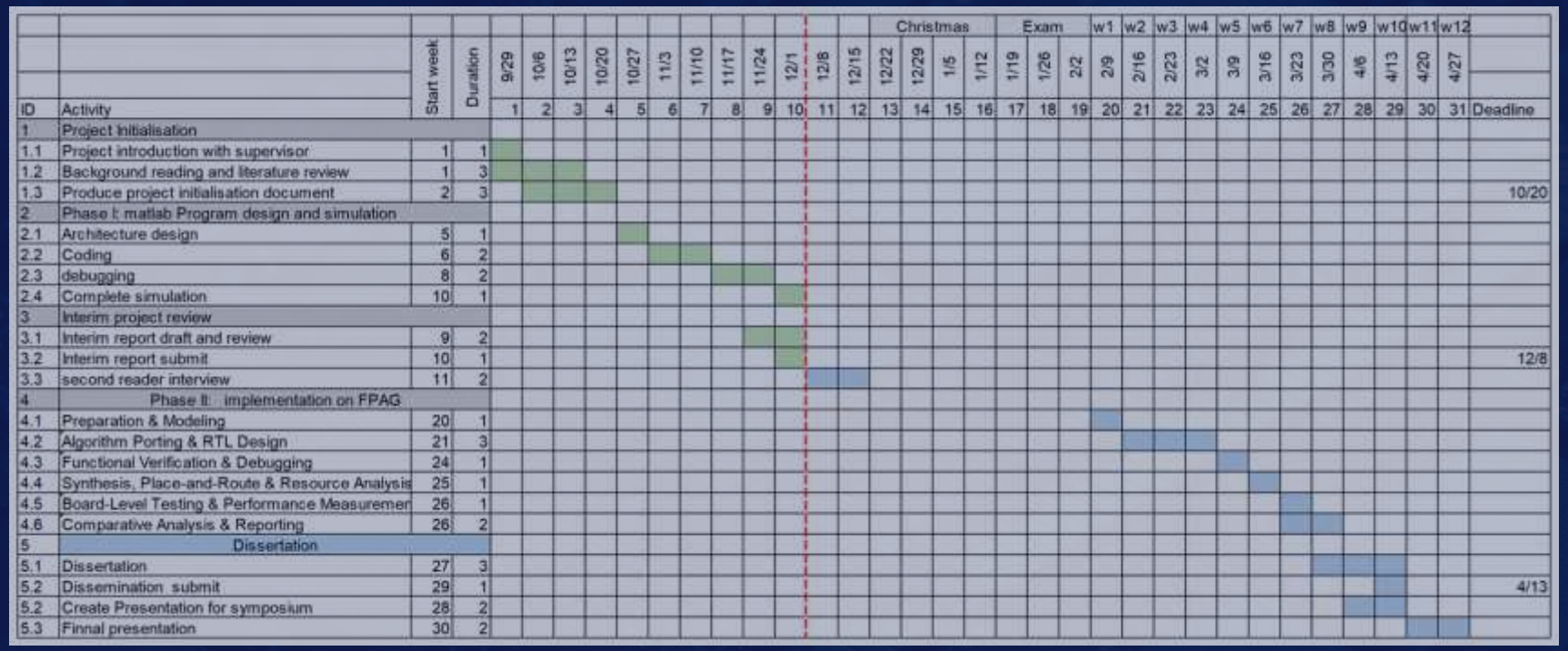
  In terms of storage efficiency, 2NRM-RRNS stands out as the most efficient, requiring minimal storage space.

- **Recommendation:**

  Given its superior storage efficiency and competitive fault tolerance, 2NRM-RRNS emerges as the most promising ECC solution for hybrid CMOS/non-CMOS memory systems.

[1] N. Z. Haron and S. Hamdioui, "Using RRNS Codes for Cluster Faults Tolerance in Hybrid Memories," IEEE Trans. Commun., 2009.

Project Plan and Future Work



S1:

- Finish the simulation
- output interim report

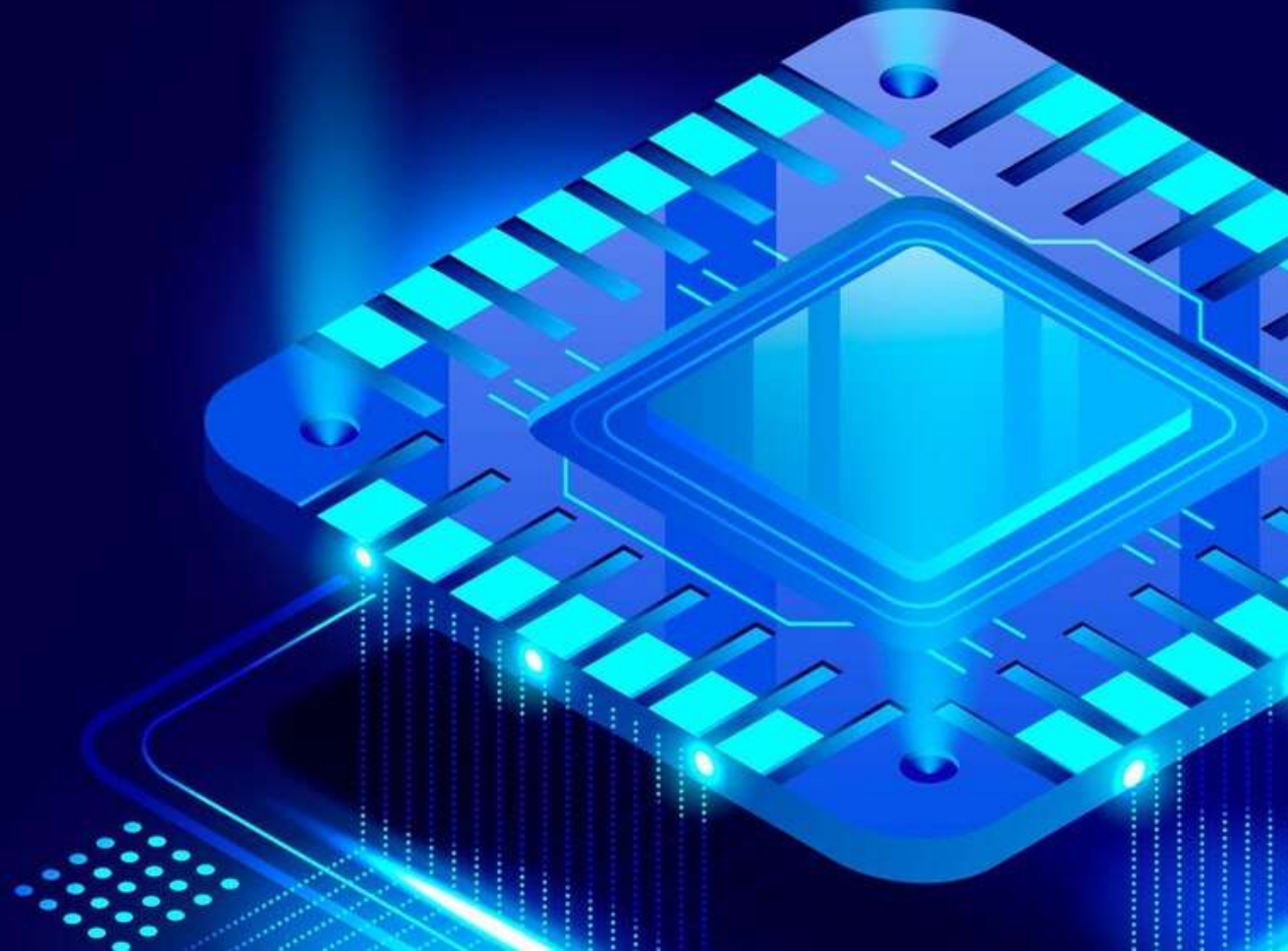
S2:

- Implement parameterizable RRNS encoder/decoder on Xilinx Artix-7 FPGA
- Measure resource utilization (LUTs, FFs) and max clock frequency
- output dissertation

Thanks

2025.10.12

Yuqi Guo



MRC Decoding of C-RRNS

Decoding Procedure

1. **Full Decoding**: Decode using all n residues to get $\hat{X}$
2. **Error Detection**:
   - If $\hat{X} \leq M_a$ (where $M_a = \prod_{i=1}^{k} m_i$), no error → output $\hat{X}$
   - Otherwise, errors detected → proceed to correction
3. **Error Correction**:
   - Select $z = n - t$ residues (t: error correction capability)
   - Compute X_z using MRC with selected residues
   - Compute $M_z = \prod_{i \in Z} m_i$ for current combination
   - Repeat for all C_n^t combinations
4. **Result Validation**:
   - If any $X_z \leq M_z$, output X_z as corrected data
   - If all $X_z > M_z$, decoding fails (uncorrectable errors)

MRC Computation Formulas

Mixed-Radix Conversion:

$$X = \sum_{s=1}^{n} v_s w_s$$

Mixed-Radix Digits:

$$v_1 = |x_1|_{m1}$$

$$v_s = \left|\left((x_s - v^1)m^1{}_s{}^{-1} - \cdots - v_{s-1}\right)m_{(s-1)s}^{-1}\right|_{ms}, \quad s = 2, \ldots, n$$

Weight Coefficients:

$$w_1 = 1$$

$$w_s = \prod_{i=1}^{s-1} m_i, s = 2, \ldots, n$$

Decoding of RRNS: Integrated MLD-Hamming Distance

Step 1: Initial Decoding Attempt

Decode using standard RRNS procedures

Check if result falls within legitimate range [0, M_a]

Step 2: Ambiguity Detection

Identify multiple valid candidates when standard decoding fails

Generate all potential valid data values within legitimate range

Step 3: Candidate Evaluation

For each valid candidate $X_{candidate}$:

Compute expected residues: $R_{expected}$ = {|X_candidate|*m1, ...,* |$X_{candidate}$|mn}

Calculate Hamming distance: $d_H(R_{received}, R_{expected})$

Record distance measurement

Step 4: Optimal Selection

Select candidate with minimum Hamming distance

MLD Objective Function

$$X_{correct} = argmin_{X \in Valid_{S}et} \Sigma_{i=1}^{n} I\left(r_{i_r eceived} \neq |X|_m i\right)$$

Residue Comparison

$$Difference_i = 0 \; if r_{i_r eceived} = |X|_m i, 1 \; otherwise$$

Distance Minimization

$$subject \; to \; X \; \in \; [0, M_a] minimize \Sigma_{i=1}^{n} \quad Difference_i$$